

Matthew Athanasopoulos (20976490) - NE451 Lab 2

November 5th, 2025

(Q1) - 10.5.1 — Antiferromagnet Simulation

- Reproduce the equivalent of Figure 10.2 from the PDF for an antiferromagnetic system ($J=-1$).

Requirements:

- Modify the sign of J in the provided code.
- Run simulations for several temperatures T .
- Plot average magnetization $\langle M \rangle$ and energy $\langle E \rangle$ vs. T
- Compare behavior to the ferromagnetic case ($J=+1$).

```
In [19]: import numpy as np
from numpy.random import rand, randint, permutation
import matplotlib.pyplot as plt
from numba import njit

# Functions from the Magnetic Phase Transitions paper (with njit
# compilation)

@njit
def ind2sub(indx, L):
    column = np.floor(indx / L)
    row = indx - column * L
    return row, column

@njit
def Metropolis(rowsub, colsub, trial, Spins, eqbmstart, N, T, L, J):
    inverseT = 1/T
    Energy = 0.0
    for counter in range(N): # for every spin site
        r = int(rowsub[counter])
        lo = r - 1
        if lo < 0:
            lo = L - 1
        hi = r + 1
        if hi > L - 1:
            hi = 0
        c = int(colsub[counter])
        lf = c - 1
```

```

        if lf < 0:
            lf = L - 1
        rt = c + 1
        if rt > L - 1:
            rt = 0
        SS = Spins[lo, c] + Spins[hi, c] + Spins[r, lf] + Spins[r, rt]
        DeltaE = 2 * J * SS * Spins[r, c]
        if DeltaE < 0 or rand() < np.exp(-DeltaE * inverseT):
            Spins[r, c] = -Spins[r, c]
        if trial >= eqbmstart:
            Energy = Energy - 0.5 * J * SS * Spins[r, c]
    return Energy

@njit
def RunMonteCarloTrials(Spins, eqbmstart, neq, trials, N, T, L, J):
    M = np.zeros(neq)
    E = np.zeros(neq)
    for trial in range(trials):
        indx = permutation(N)
        rowsub, colsub = ind2sub(indx, L)
        Energy = Metropolis(rowsub, colsub, trial, Spins, eqbmstart,
                             N, T, L, J)
        if trial >= eqbmstart:
            t = trial - eqbmstart
            E[t] = Energy
            M[t] = np.sum(Spins)
    Eav = np.mean(E) / N
    Cv = (np.var(E) / T**2) / N
    Mav = np.mean(M) / N
    Chi = (np.var(M) / T) / N
    return [Eav, Cv, Mav, Chi]

```

In [25]: *# helper functions for q1*

```

def new_random_spins(L):
    """Generate a fresh LxL spin configuration with values in {+1, -1}."""
    return -1 + 2 * randint(2, size=(L, L))

def run_ising_grid(temps, L, eqbmstart, neq, averages, J):
    """
    Run multiple independent sweeps across a grid of temperatures.
    Returns an array with shape (len(temps), averages, 4) for [E, Cv, M, Chi]
    """
    N = L * L
    total_trials = eqbmstart + neq
    data = np.empty((len(temps), averages, 4), dtype=float)

    for rep in range(averages):
        spins = new_random_spins(L)
        for i, T in enumerate(temps):
            data[i, rep] = RunMonteCarloTrials(spins, eqbmstart, neq,
                                                total_trials, N, T, L, J)

    return data

def plot_multi_panel(raw_data, temps, super_title):
    """

```

```

Four subplots: <E>, C_v, <M>, Chi for all repeats + the average curve.
AFM only (J = -1) per assignment.
''''''
avg = raw_data.mean(axis=1) # (len(T), 4)
fig, axs = plt.subplots(2, 2, figsize=(14, 9))
fig.subplots_adjust(hspace=0.32, wspace=0.26)
labels = [r'$\langle E \rangle$', r'$C_V$', r'$\langle M \rangle$', r'$\chi$']
idxs = [(0,0),(0,1),(1,0),(1,1)]

for rep in range(raw_data.shape[1]):
    axs[0,0].plot(temps, raw_data[:, rep, 0], alpha=0.35, linewidth=1)
    axs[0,1].plot(temps, raw_data[:, rep, 1], alpha=0.35, linewidth=1)
    axs[1,0].plot(temps, raw_data[:, rep, 2], alpha=0.35, linewidth=1)
    axs[1,1].plot(temps, raw_data[:, rep, 3], alpha=0.35, linewidth=1)
    axs[0,0].plot(temps, avg[:,0], linewidth=2, label='mean')
    axs[0,1].plot(temps, avg[:,1], linewidth=2, label='mean')
    axs[1,0].plot(temps, avg[:,2], linewidth=2, label='mean')
    axs[1,1].plot(temps, avg[:,3], linewidth=2, label='mean')

for ax, lab in zip([axs[0,0],axs[0,1],axs[1,0],axs[1,1]], labels):
    ax.set_xlabel("Temperature $T$")
    ax.set_ylabel(lab)
    ax.grid(True, alpha=0.3)
    ax.legend(loc='best')

fig.suptitle(super_title)
plt.show()

```

```

In [29]: # Grid & run-params
L        = 100
num_points = 30
num_repeats = 5
T_min, T_max = 1.0, 4.0
temps      = np.linspace(T_min, T_max, num_points)
eqbmstart  = 2000
neq        = 1000
J_AFM     = -1

# Run anti-ferromagnetic simulations across T
afm_data = run_ising_grid(temps=temps, L=L, eqbmstart=eqbmstart, neq=neq,
                          averages=num_repeats, J=J_AFM)

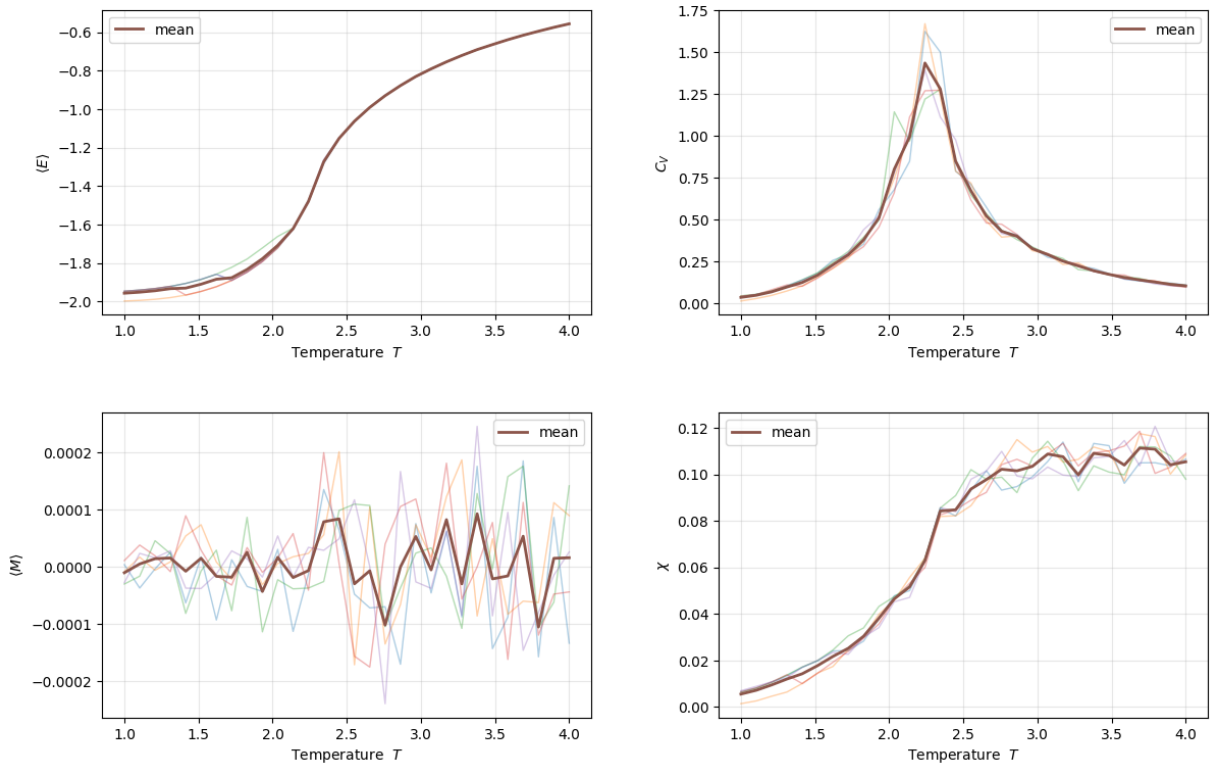
```

```

In [37]: # This plots each repeat faintly and overlays the bold mean curve vs. T
# (mean is computed across repeats inside plot_multi_panel as raw_data.mean(
plot_multi_panel(
    raw_data=afm_data,
    temps=temps,
    super_title="2D Ising (J = -1) - Antiferromagnet: <E>, C_V, <M>, χ vs T"
)

```

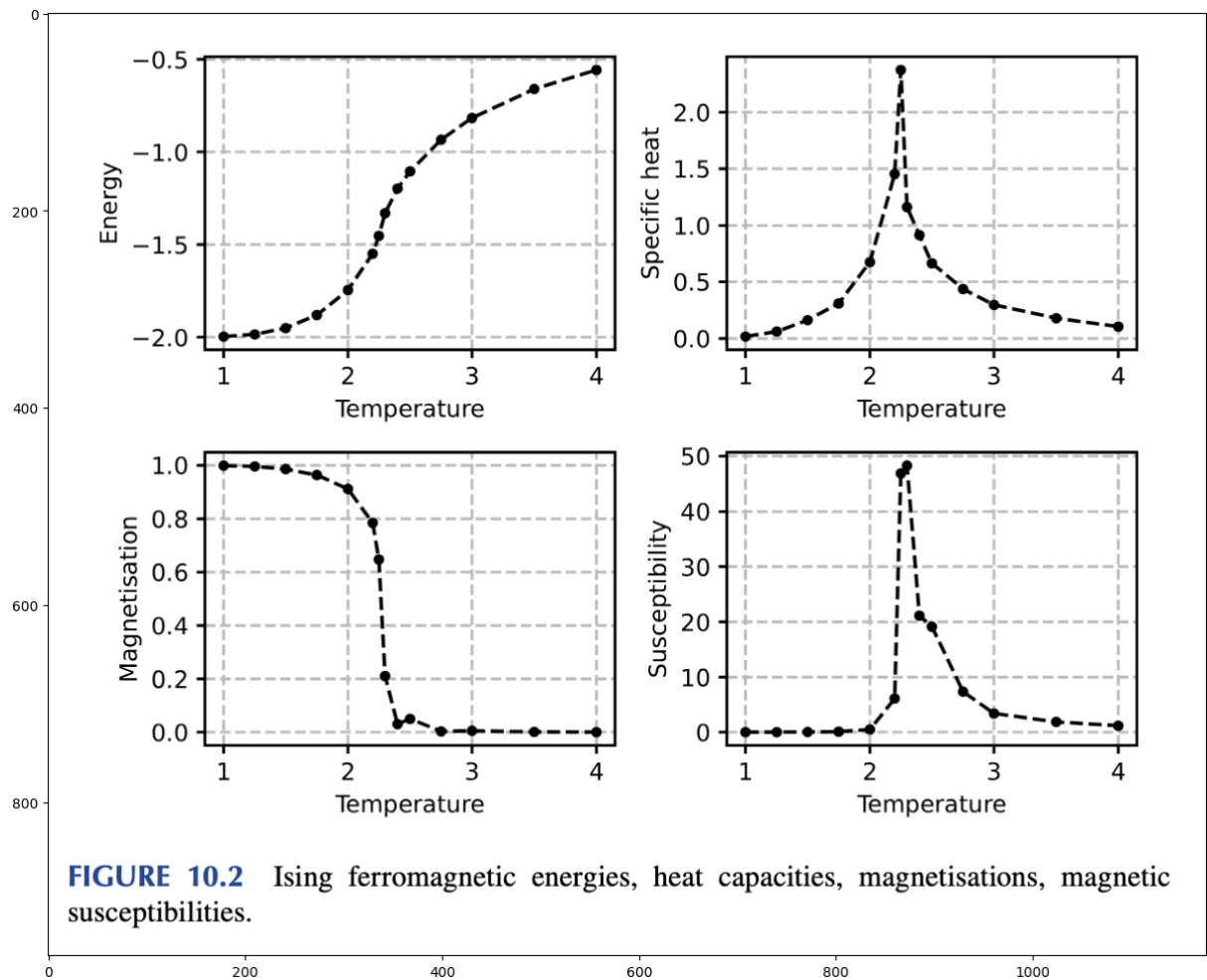
2D Ising ($J = -1$) - Antiferromagnet: $\langle E \rangle$, C_V , $\langle M \rangle$, χ vs T



We're comparing to the below plot (from the paper)

```
In [38]: import matplotlib.image as mpimg
img = mpimg.imread("compare.png")
plt.figure(figsize=(15, 15))
plt.imshow(img)
```

```
Out[38]: <matplotlib.image.AxesImage at 0x10e0dc500>
```



Comments

Energy and heat capacity match between the antiferromagnet ($J = -1$) and the ferromagnet ($J = +1$) on a square bipartite lattice with no external field. They are related by a sublattice spin flip, so the average energy $\langle E \rangle(T)$ and the heat capacity $C_v(T)$ have essentially the same temperature dependence apart from finite-size fluctuations. Both show a clear C_v peak near the critical region.

For the antiferromagnet, the uniform average magnetization $\langle M \rangle$ stays near zero because the low-temperature state is checkerboard ordered with equal up and down spins on opposite sublattices. In the ferromagnet, the system develops spontaneous magnetization below T_c , so $|\langle M \rangle|$ is large at low temperature and then falls toward zero as temperature passes through the phase transition.

The uniform susceptibility $\chi = \text{Var}(M)/T$ is correspondingly small and relatively flat for the antiferromagnet, while the ferromagnet's uniform χ is much larger and peaks near T_c . If one measured staggered quantities for the antiferromagnet (staggered magnetization and susceptibility), those would exhibit ordering and a peak analogous to the ferromagnet's uniform responses.

The apparent peak location in C_v (and in the ferromagnet's χ) occurs at

the same T_c for antiferromagnet and ferromagnet on this lattice. Finite lattice size and averaging round the singular behavior into smooth, broadened peaks. In the plotted panel, faint lines show individual repeats versus temperature and the bold line overlays their mean, which makes the overall trend clearer while de-emphasizing run-to-run noise.

(Q2) - 10.5.2 — Equilibrium Analysis

- Investigate whether 2000 sweeps are sufficient to reach equilibrium.

Steps:

- Choose temperatures below, near, and above the phase transition (e.g. $T=1.5, 2.3, 3.0$).
- Track energy and magnetization as functions of the number of sweeps.
- Plot E and M vs. sweeps to see when the system stabilizes.
- Comment on whether 2000 sweeps are sufficient.

```
In [49]: def new_random_spins(L):
          return -1 + 2 * randint(2, size=(L, L))

def energy_per_spin(spins, J):
    """
    Instantaneous energy per spin using periodic BCs.
    Equivalent to  $-J * \sum_{\langle i,j \rangle} s_i s_j / N$ .
    """
    # nearest neighbors via roll
    nn = (
        np.roll(spins, 1, axis=0) +
        np.roll(spins, -1, axis=0) +
        np.roll(spins, 1, axis=1) +
        np.roll(spins, -1, axis=1)
    )
    E = -J * 0.5 * np.sum(spins * nn)
    return E / spins.size

def magnetization_per_spin(spins):
    return np.sum(spins) / spins.size

def run_equilibrium_check(T, L, J, total_sweeps, stride=5, seed=None):
    """
    Run Metropolis sweeps and record E/N and M/N every `stride` sweeps
    without modifying the njit kernels. Unique approach: checkpoint sampling
    with explicit configuration energy.
    """
    if seed is not None:
        np.random.seed(seed)
    N = L * L
    spins = new_random_spins(L)
    nsteps = total_sweeps // stride + 1
```

```

E_series = np.empty(nsteps, dtype=float)
M_series = np.empty(nsteps, dtype=float)
sweeps_axis = np.arange(0, total_sweeps + 1, stride, dtype=int)

# initial measurement (sweep 0)
E_series[0] = energy_per_spin(spins, J)
M_series[0] = magnetization_per_spin(spins)

# evolve + checkpoint
write_idx = 1
for sweep in range(1, total_sweeps + 1):
    # one full sweep via our existing kernel
    indx = permutation(N)
    rowsub, colsub = ind2sub(indx, L)
    _ = Metropolis(rowsub, colsub, sweep, spins, 0, N, T, L, J)
    if sweep % stride == 0:
        E_series[write_idx] = energy_per_spin(spins, J)
        M_series[write_idx] = magnetization_per_spin(spins)
        write_idx += 1

return sweeps_axis, E_series, M_series

def run_repeats(T_list, L, total_sweeps, stride, repeats, J):
    """
    For each temperature, run several independent repeats and return stacked
    """
    out = {}
    for T in T_list:
        curves = []
        for r in range(repeats):
            s, E, M = run_equilibrium_check(T=T, L=L, J=J, total_sweeps=total_sweeps,
                                           stride=stride, seed=None)
            curves.append((E, M))
        # stack shape: (repeats, nsteps)
        E_stack = np.vstack([c[0] for c in curves])
        M_stack = np.vstack([c[1] for c in curves])
        out[T] = (s, E_stack, M_stack)
    return out

def plot_equilibrium_panels(results_afm, results_fm, vline_at=None, title_su
    """
    For each T, plot E/N and M/N vs sweeps.
    Faint lines = repeats; bold = mean across repeats.
    """
    Ts = list(results_afm.keys())
    nT = len(Ts)
    fig, axes = plt.subplots(nT, 2, figsize=(14, 4.2*nT), sharex=True)
    if nT == 1:
        axes = np.array([axes])

    for row, T in enumerate(Ts):
        # AFM
        s, E_a, M_a = results_afm[T]
        # FM
        _, E_f, M_f = results_fm[T]

```

```

# left: Energy
axE = axes[row, 0]
for k in range(E_a.shape[0]):
    axE.plot(s, E_a[k], alpha=0.35, linewidth=1, label=None)
for k in range(E_f.shape[0]):
    axE.plot(s, E_f[k], alpha=0.35, linewidth=1, label=None)
axE.plot(s, E_a.mean(axis=0), linewidth=2, label="AFM mean (J=-1)")
axE.plot(s, E_f.mean(axis=0), linewidth=2, label="FM mean (J=+1)")
axE.set_ylabel("E/N")
axE.set_title(f"T = {T} - Energy vs sweeps")
axE.grid(True, alpha=0.3)
axE.axvline(vline_at, linestyle="--", alpha=0.8)
axE.legend(loc="best")

# right: Magnetization
axM = axes[row, 1]
for k in range(M_a.shape[0]):
    axM.plot(s, M_a[k], alpha=0.35, linewidth=1, label=None)
for k in range(M_f.shape[0]):
    axM.plot(s, M_f[k], alpha=0.35, linewidth=1, label=None)
axM.plot(s, M_a.mean(axis=0), linewidth=2, label="AFM mean (J=-1)")
axM.plot(s, M_f.mean(axis=0), linewidth=2, label="FM mean (J=+1)")
axM.set_ylabel("M/N")
axM.set_title(f"T = {T} - Magnetization vs sweeps")
axM.grid(True, alpha=0.3)
if vline_at is not None:
    axM.axvline(vline_at, linestyle="--", alpha=0.8)

axM.legend(loc="best")

axes[-1, 0].set_xlabel("Sweeps")
axes[-1, 1].set_xlabel("Sweeps")
fig.suptitle(f"Equilibrium analysis (checkpoint sampling){title_suffix}")
plt.show()

```

```

In [59]: T_list      = [1.5, 2.3, 3.0]
L        = 100
total_sweeps = 8000 # long run to visualize stabilization
stride     = 5 # record every 5 sweeps
repeats    = 3 # a few repeats for variability

```

```

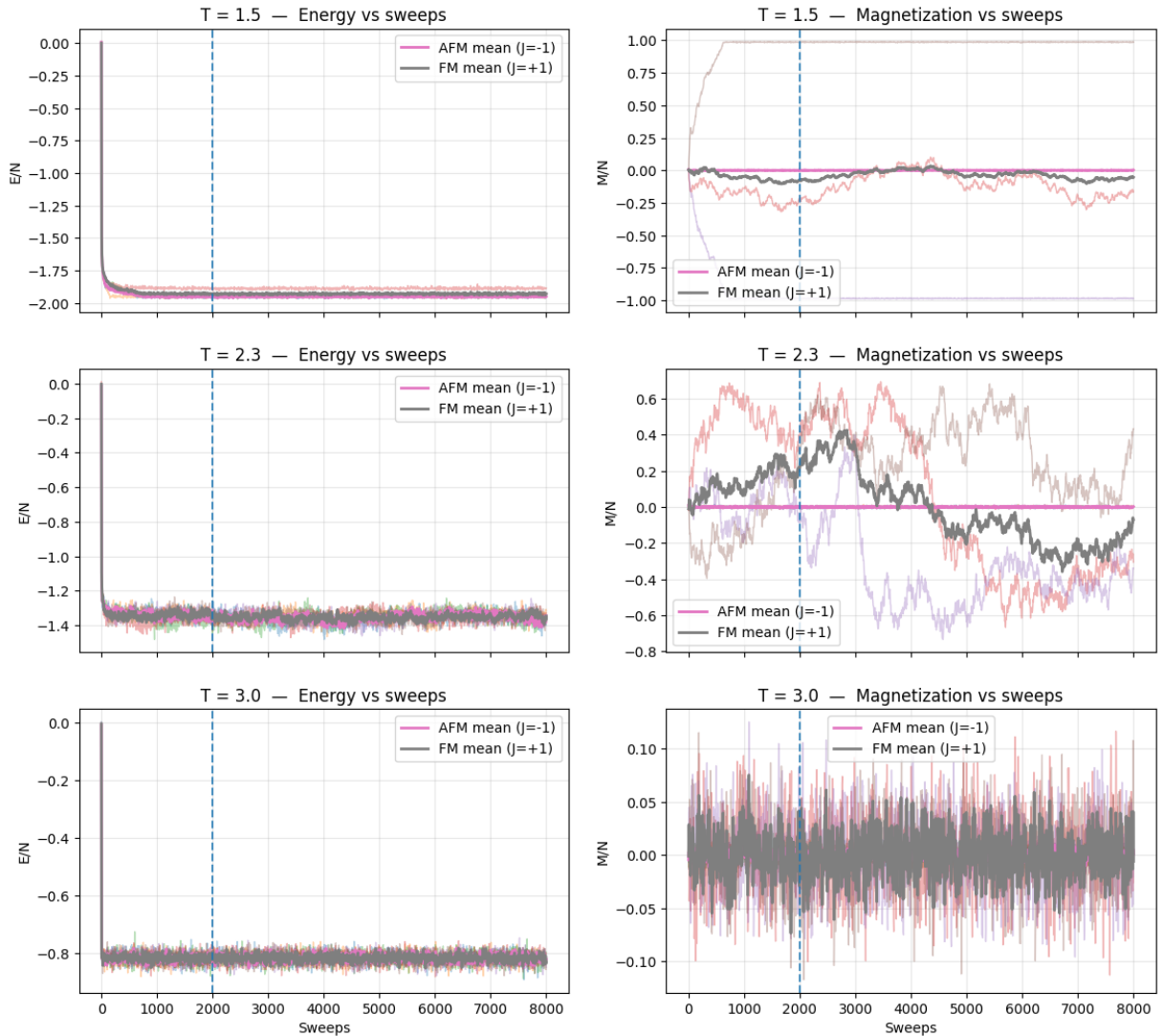
# AFM and FM runs
results_afm = run_repeats(T_list, L=L, total_sweeps=total_sweeps,
                           stride=stride, repeats=repeats, J=-1)
results_fm  = run_repeats(T_list, L=L, total_sweeps=total_sweeps,
                           stride=stride, repeats=repeats, J=+1)

```

```

In [61]: # This plots faint per-repeat curves and overlays bold means.
# The dashed vertical line marks 2000 sweeps to assess sufficiency.
plot_equilibrium_panels(results_afm, results_fm, vline_at=2000,
                        title_suffix=" - AFM vs FM")

```



Comments

Left panels are energy per spin versus sweeps; right panels are magnetization per spin versus sweeps. Faint traces are individual repeats and the bold curves are the mean across repeats. The dashed vertical line marks 2000 sweeps.

From the energy panels, equilibration is fast at all three temperatures. There is a brief initial drop and then only small fluctuations; by the 2000-sweep mark the curves have essentially flattened for both AFM and FM. The AFM and FM energy means lie almost on top of each other, which is expected on a bipartite lattice with zero field.

From the magnetization panels, behavior depends on temperature and phase. For the AFM, the uniform magnetization stays near zero and stabilizes quickly at all T , so 2000 sweeps is ample. For the FM at $T = 1.5$ (below T_c), some repeats quickly choose a magnetized state while others coarsen more slowly or wander; the mean can appear close to zero and

deceptively stable, but that is only because different runs sit near +1 or -1 and average out. In those wandering/sign-splitting cases, 2000 sweeps is not enough to claim a stable magnetization estimate, even though energy has equilibrated. Near the transition at $T = 2.3$, critical slowing down is visible: FM magnetization shows long drifts and occasional sign changes, making 2000 sweeps marginal. Above the transition at $T = 3.0$, both AFM and FM magnetizations fluctuate around zero with short correlation times; here 2000 sweeps is sufficient.

Overall, 2000 sweeps are adequate for energy at all three temperatures and for magnetization away from the critical region. Near and below T_c in the ferromagnet, longer runs and/or more repeats are advisable to average over slow domain dynamics and late sign flips.

(Q3) - 10.5.3 — External Magnetic Field

Modify the provided code to include a magnetic field term.

Tasks:

- Implement this in your code.
- Run simulations at various temperatures.
- Compute and plot:
 - - Energy $\langle E \rangle$
 - - Magnetization $\langle M \rangle$
 - - Heat capacity CV
 - - Susceptibility χ
- Compare with the zero-field case and discuss the effect of H .

```
In [62]: from numba import njit
import numpy as np
from numpy.random import permutation

@njit
def Metropolis_H(rowsub, colsub, trial, Spins, eqbmstart, N, T, L, J, h):
    """
    Metropolis sweep with uniform external field h = g*mu_B*H.
    Uses  $\Delta E = 2 s_{rc} (J * S_{nn} + h)$ .
    Accumulates energy after the (possible) flip:
         $E += -0.5 * J * s_{rc} * S_{nn} - h * s_{rc}$ 
    """
    invT = 1.0 / T
    Energy = 0.0
    for counter in range(N):
        r = int(rowsub[counter])
```

```

    lo = r - 1
    if lo < 0: lo = L - 1
    hi = r + 1
    if hi > L - 1: hi = 0
    c = int(colsub[counter])
    lf = c - 1
    if lf < 0: lf = L - 1
    rt = c + 1
    if rt > L - 1: rt = 0

    # nearest-neighbour sum
    SS = Spins[lo, c] + Spins[hi, c] + Spins[r, lf] + Spins[r, rt]

    # field-aware Metropolis
    DeltaE = 2.0 * Spins[r, c] * (J * SS + h)
    if DeltaE < 0.0 or np.exp(-DeltaE * invT) > np.random.rand():
        Spins[r, c] = -Spins[r, c]

    if trial >= eqbmstart:
        Energy += -0.5 * J * SS * Spins[r, c] - h * Spins[r, c]

    return Energy

@jit
def RunMonteCarloTrials_H(Spins, eqbmstart, neq, trials, N, T, L, J, h):
    """
    Return [⟨E⟩, C_v, ⟨M⟩, χ] for a given T, J, field h.
    Shape/semantics match your Q1 RunMonteCarloTrials for drop-in plotting.
    """
    M = np.zeros(neq)
    E = np.zeros(neq)

    for trial in range(trials):
        indx = permutation(N)
        rowsub, colsub = ind2sub(indx, L) # uses your existing ind2sub
        Energy = Metropolis_H(rowsub, colsub, trial, Spins, eqbmstart, N, T,
        if trial >= eqbmstart:
            t = trial - eqbmstart
            E[t] = Energy
            M[t] = np.sum(Spins)
    Eav = np.mean(E) / N
    Cv = (np.var(E) / (T**2)) / N
    Mav = np.mean(M) / N
    Chi = (np.var(M) / T) / N
    return [Eav, Cv, Mav, Chi]

```

In [63]:

```

def run_ising_grid_with_field(temps, L, eqbmstart, neq, averages, J, h):
    """
    Like your run_ising_grid, but uses the field-aware kernel with field h.
    Returns array of shape (len(temps), averages, 4) for [E, Cv, M, Chi].
    """
    N = L * L
    trials = eqbmstart + neq
    out = np.empty((len(temps), averages, 4), dtype=float)
    for rep in range(averages):
        spins = -1 + 2 * np.random.randint(2, size=(L, L))

```

```

        for i, T in enumerate(temps):
            out[i, rep] = RunMonteCarloTrials_H(spins, eqbmstart, neq, trial)
    return out

def plot_compare_field(measured_h1, measured_h0, temps, title):
    """
    Overlay means for h=1 and h=0 in your 2x2 layout.
    """
    mean_h1 = measured_h1.mean(axis=1) # (len(T), 4)
    mean_h0 = measured_h0.mean(axis=1)

    fig, axs = plt.subplots(2, 2, figsize=(14, 9))
    fig.subplots_adjust(hspace=0.32, wspace=0.26)
    labels = [r'$\langle E \rangle$', r'$C_V$', r'$\langle M \rangle$', r'$\langle S \rangle$']
    slots = [(0,0),(0,1),(1,0),(1,1)]

    for k, (r,c) in enumerate(slots):
        axs[r,c].plot(temps, mean_h0[:,k], linewidth=2, label='h = 0')
        axs[r,c].plot(temps, mean_h1[:,k], linewidth=2, label='h = 1')
        axs[r,c].set_xlabel("Temperature $T$")
        axs[r,c].set_ylabel(labels[k])
        axs[r,c].grid(True, alpha=0.3)
        axs[r,c].legend(loc='best')

    fig.suptitle(title)
    plt.show()

```

```

In [64]: # grid & params (same style as Q1)
L        = 100
temps    = np.linspace(1.0, 4.0, 30)
eqbmstart = 2000
neq       = 1000
averages  = 5

J = -1      # AFM; set J = +1 to repeat for FM
h1 = 1.0    # g*mu_B*H
h0 = 0.0

# zero-field
data_h0 = run_ising_grid(temps=temps, L=L, eqbmstart=eqbmstart, neq=neq,
                        averages=averages, J=J)

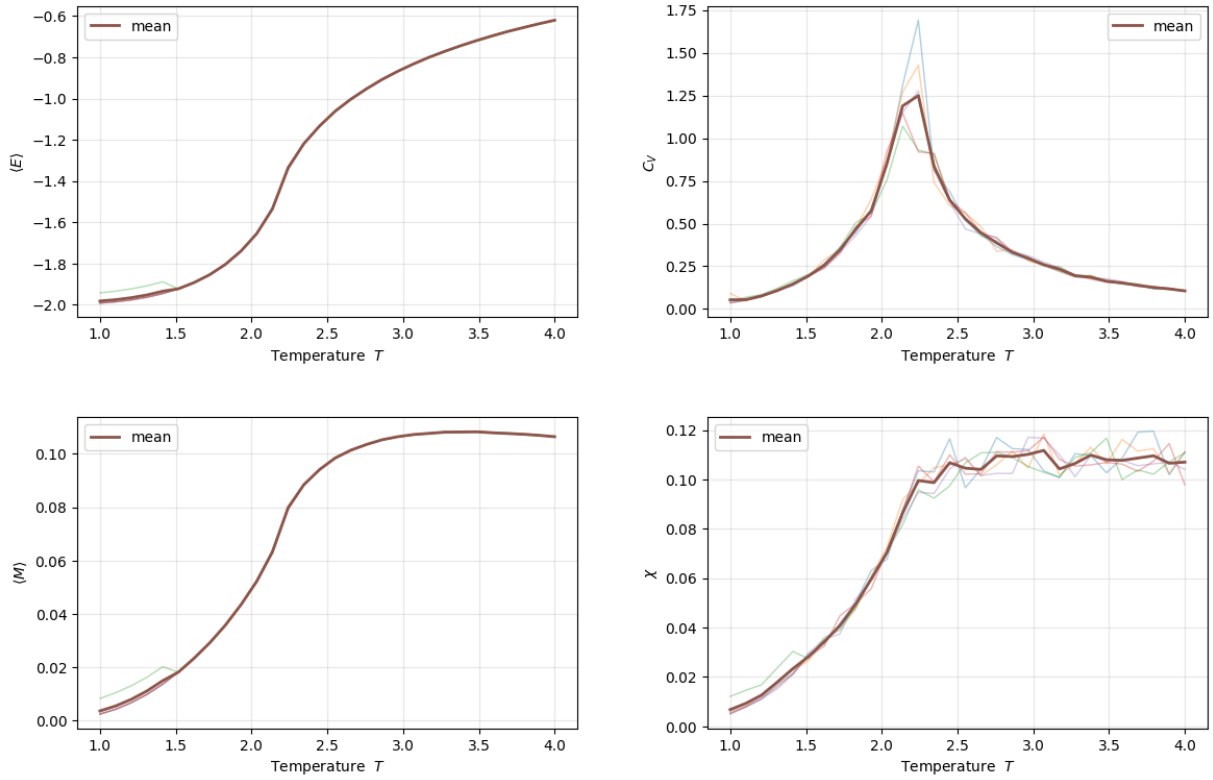
# with field
data_h1 = run_ising_grid_with_field(temps=temps, L=L, eqbmstart=eqbmstart, neq=neq,
                                averages=averages, J=J, h=h1)

# individual panels
plot_multi_panel(data_h1, temps, super_title=f"2D Ising with field (h = 1),

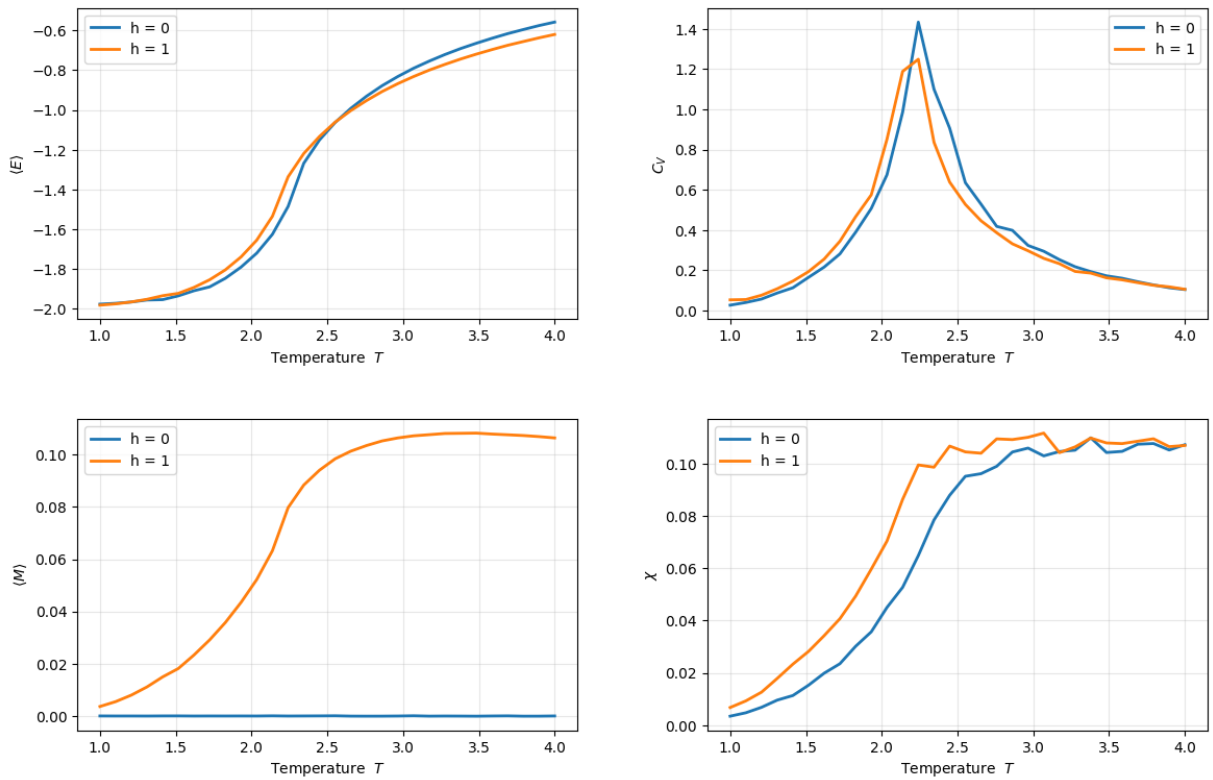
# side-by-side comparison of means
plot_compare_field(data_h1, data_h0, temps, title=f"Effect of field h=1 vs h

```

2D Ising with field ($h = 1$), $J = -1$



Effect of field $h=1$ vs $h=0$ ($J = -1$)



Comments

For antiferromagnetic ($J = -1$), field on (first 2×2): With $h = 1$ the energy increases smoothly with temperature and is less negative than at low T , with a noticeable curvature change around $T \approx 2.1$ – 2.3 . The heat capacity shows a single pronounced peak near that temperature range; across repeats the peak height varies a bit but the mean curve is clean. The uniform magnetization rises monotonically from ~ 0 at low T to ≈ 0.11 by $T \approx 3$, reflecting a net polarization induced by the field even in an antiferromagnet. The uniform susceptibility climbs rapidly near the same temperature range and then levels off into a broad plateau around ~ 0.11 ; at high T it slowly decays, consistent with Curie-like behavior.

Comparison $h = 1$ (orange) vs $h = 0$ (blue) for $J = -1$ (second 2×2): The field doesn't majorly shift the mean energy. The heat-capacity peak is lower and very slightly broader with $h = 1$, which may be expected when the zero-field singularity is rounded into a crossover; any slight shift in the apparent peak position is within what you'd expect from rounding and finite size. The clearest change is in the uniform magnetization: it is ≈ 0 for all T when $h = 0$, but becomes strictly positive and increases smoothly with T under $h = 1$, saturating a little above 0.1 . The uniform susceptibility with $h = 1$ rises earlier and peaks at a similar amplitude to $h = 0$. Because a nonzero field breaks the up–down symmetry, the 2D Ising model no longer has a true zero-field critical singularity: the susceptibility stays finite, exhibits a rounded maximum associated with a smooth crossover, and would eventually decay at high temperature.